COMPUTER GRAPHICS LAB REPORT

LAB 02: BASIC 2D DRAWING WITH TURTLE GRAPHICS

COURSE: COMPUTER GRAPHICS LAB

SUBMITTED BY -

SHAHRIAR ALOM MASUD

STUDENT ID: 2201032

SESSION: 2022–2023

SUPERVISED BY -

FARZANA AKTER

ASSISTANT PROFESSOR

DEPARTMENT OF IRE, UFTB



FACULTY OF CYBER PHYSICAL SYSTEM ENGINEERING

DEPARTMENT OF INTERNET OF THINGS AND ROBOTICS ENGINEERING (IRE)

UNIVERSITY OF FRONTIER TECHNOLOGY, BANGLADESH

OCTOBER 2026

Task 1: Drawing a Smiling Face

GitHub Repository Link:

https://github.com/Masud744/Computer-Graphics-Lab/blob/main/task1.py

Python Source Code (task1.py):

```python
from turtle import *

speed(3)

# 1. Main face outline (radius 300 circle)
pensize(3)
pencolor("black")
penup()
goto(0, -300)
pendown()
circle(300)

# 2. Smiling mouth
# Red blot first, then a white blot slightly above it to make a crescent
penup()
goto(0, -90)
pendown()
dot(180, "red")

penup()
goto(0, -65)
pendown()
dot(180, "white")

# 3. Nose (thick blue triangle)
# Drawn after mouth so the white blot doesn't cover it
pensize(5)
pencolor("blue")
penup()
goto(0, 30)
pendown()
goto(-35, -40)
goto(35, -40)
goto(0, 30)

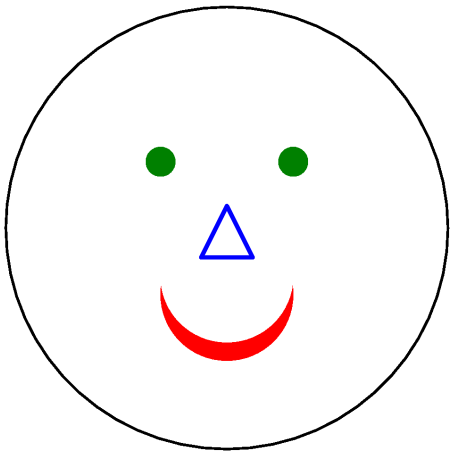
# 4. Eyes (small green filled circles)
penup()
goto(-90, 90)
pendown()
dot(40, "green")

penup()
goto(90, 90)
pendown()
dot(40, "green")

hideturtle()
done()
```

Problem Description: Draw a face enclosed in a black circle (radius 300), small green blots for eyes, a thick blue triangle for a nose, and a red smiling mouth. The smiling mouth is formed by drawing a red blot, and then drawing a white blot slightly higher to leave a crescent of red.

Output Visualization:



Key Observation / Discussion: The order of operations is crucial. Because the white blot creates the crescent smile by partially covering the underlying shape, the mouth must be drawn **before** the nose and eyes. Drawing the mouth afterward would overwrite the blue triangle nose with white.

Task 2: Flag Rectangle and Proportions (10:6)

GitHub Repository Link:

https://github.com/Masud744/Computer-Graphics-Lab/blob/main/task2.py

Python Source Code (task2.py):

```
from turtle import *

speed(3)

# Flag dimensions with 10:6 ratio
length = 600
height = 360

# 1. Green background rectangle
penup()
goto(-300, -180)
pendown()
color("green", "green")
begin_fill()
for i in range(2):
    forward(length)
    left(90)
    forward(height)
    left(90)
end_fill()

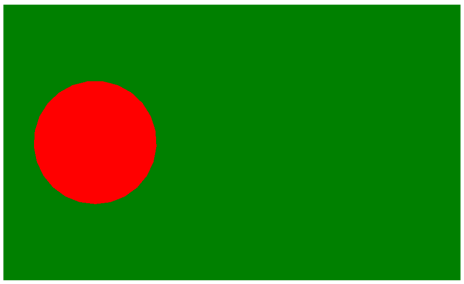
# 2. Red circle inside
# Circle centre is placed at 1/5 of the length from the
    left edge (-300 + 600/5 = -180)
circle_center_x = -300 + (length / 5)
radius = 80

penup()
goto(circle_center_x, -radius)
pendown()
color("red", "red")
begin_fill()
circle(radius)
end_fill()

hideturtle()
done()
```

Problem Description: Draw a green background rectangle with proportions of approximately 10:6 (length). Draw a red circle inside the rectangle. Fill the rectangle with green and the circle with red. Position the centre of the red circle at one-fifth (1/5) of the rectangle's length from the left edge.

Output Visualization:



Observation / Analysis: In the authentic National Flag of Bangladesh, the centre of the red circle is located at 9/20 (45%) of the length from the left edge. In this lab task, placing the circle at 1/5 (20%) from the left edge visibly shifts the circle to the far-left border, which was the intended observation.

Task 3: Overlapping Concentric Squares (8-Point Star)

GitHub Repository Link:
https://github.com/Masud744/Computer-Graphics-Lab/blob/main/task3.py

Python Source Code (`task3.py`):

```python
import math
from turtle import *

speed(3)
pensize(2)

size = 200

# 1. First square (straight/upright)
penup()
goto(-size / 2, -size / 2)
pendown()
for i in range(4):
    forward(size)
    left(90)

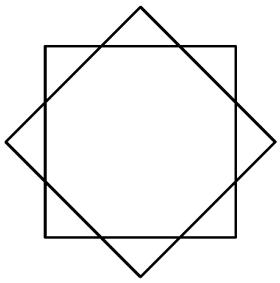
# 2. Second square (rotated by 45 degrees - diamond shape)
# Distance from center (0,0) to corner vertex along axes = (size / 2) * sqrt(2)
diagonal_offset = (size / 2) * math.sqrt(2)

penup()
goto(0, -diagonal_offset)
setheading(45)
pendown()
for i in range(4):
    forward(size)
    left(90)

hideturtle()
done()
```

Problem Description: Draw the shape consisting of two overlapping squares sharing the identical center point. One square is aligned horizontally/vertically ($0°$), and the second square is rotated by $45°$, forming an 8-pointed star shape.

Output Visualization:



Geometric Explanation: Both squares have equal side length $S = 200$ and center at $(0, 0)$. The straight square is drawn from corner $(-S/2, -S/2)$. For the $45°$ rotated square, the distance from center to corner along the vertical axis is $R = \frac{S}{\sqrt{2}}$. Starting at $(0, -R)$ with heading $45°$ yields the exact diamond orientation.

Task 4: Elegant Rotated Squares Pattern

GitHub Repository Link:

https://github.com/Masud744/Computer-Graphics-Lab/blob/main/task4.py

Python Source Code (task4.py):

```python
from turtle import *

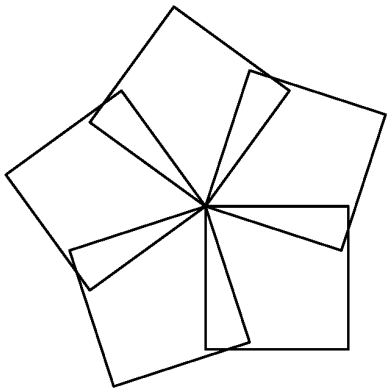
speed(3)
pensize(2)

# Drawing 5 squares rotated around a single corner
# Total angle 360 / 5 = 72 degrees
for i in range(5):
    # Draw one square
    for j in range(4):
        forward(150)
        right(90)
    # Rotate 72 degrees for the next square
    right(72)

hideturtle()
done()
```

Problem Description: Draw the 5-square rotational pattern. Think carefully about how you might be able to draw this elegantly (efficiently and readably).

Output Visualization:



Elegance & Efficiency: The pattern exhibits 5-fold rotational symmetry about a common corner vertex. Instead of repetitive code, it is implemented using a concise nested loop:

- **Outer loop:** Repeats 5 times, turning right by $360^\circ/5 = 72^\circ$.
- **Inner loop:** Draws the 4 sides of each square with 90° turns.

This results in clean, readable, and mathematically optimal code.